

Book Matrix

Complete System Documentation & Architectural Specification

BOOK MATRIX - TECHNICAL DOCUMENTATION AND SYSTEM SPECIFICATION

1. System Overview & Philosophy

Book Matrix is a high-tech, responsive Library Management System designed to transition libraries from traditional paper records to an automated, intelligent, and highly customizable digital workspace. The interface features a premium glassmorphic visual style with dark and light theme synchronization.

2. Technology Stack & Programming Languages

The application is built on a hybrid architecture to serve both high-efficiency desktop environments and school coursework syllabus demands:

2.1 Frontend

- HTML5: Semantic architecture for layout routing (Dashboard, Books, Members, Borrow/Return, Transactions, Settings, Messages).
- CSS3 (Vanilla CSS): Implements visual tokens, glassmorphism (``backdrop-filter``), variables for dynamic themes, flexbox/grid grids, and micro-animations.
- JavaScript (ES6): Client-side dynamic scripting, fetch API endpoint integrations, routing, form validation, and Chart.js animations.

2.2 Backend & Middleware

- Python (http.server & socketserver): Light-weight local server providing static file routing and RESTful API endpoints.
- SQLite (sqlite3): Production database layer storing books, users, settings, and logs.
- C (MySQL & CGI): An auxiliary syllabus-compliant module (``api.c``) utilizing standard ``mysql.h`` to connect to a MySQL server and perform CGI actions with a custom implementation of Bubble Sort to order catalog records.

2.3 AI Layer

- Gemini API (gemini-2.5-flash): Serves as the "AI Library Assistant", bound with custom database function tools via the Google Generative AI SDK.
 - Local Fallback Engine: A regex-based offline query engine that resolves requests when the Gemini API is offline.
-

3. Step-by-Step Architectural Decisions (What we did & Why)

1. Dashboard & UI Design (Why): Created a premium glassmorphism dashboard first. Visual aesthetic represents modern web interfaces, making the application inviting and professional.
 2. Database Integration (Why): Converted hardcoded views to SQLite backend tables. SQLite is serverless, self-contained, and requires zero configuration.
 3. CGI C-Sorting Module (Why): Maintained ``api.c`` for educational compliance, showcasing how raw C structures and Bubble Sort algorithms sort database queries.
 4. AI Assistant and Tool Binding (Why): Bound the chat engine directly to database lookup functions (``db_search_books``, ``db_get_library_statistics``). This permits users to retrieve actual data using natural language.
 5. Configurable Settings (Why): Added sliders and inputs for fine rates, checkout periods, and API keys. This ensures the tool adapts to different administrative rules.
 6. Automated Backups (Why): Integrated backup schedulers to archive snapshots to protect system data.
 7. Cache Busting (Why): Appended version query strings to files to prevent the browser from displaying stale CSS or JS assets.
-

4. Backend Endpoints (API Specification)

4.1 GET Routes (``/api?action=...``)

- ``dashboard``: Gathers general numbers (titles, copies, registers, overdue books, borrows).
- ``get_books``: Searches and filters the book database.
- ``get_categories`` / ``get_authors`` / ``get_publishers``: Returns lists of attributes for dropdown filters.
- ``get_book_details``: Retrieves details of a specific book by ID.
- ``get_users``: Displays member directories.
- ``get_settings``: Returns administrative thresholds.
- ``get_backups``: Lists available SQLite backup dates.
- ``get_audit_logs``: Retrieves internal logs.

4.2 POST Routes (``/api?action=...``)

- ``issue_book``: Checks out a copy to a member if conditions are met.
 - ``return_book``: Returns a borrowed copy and calculates overdue fines.
 - ``db_backup`` / ``db_restore`` / ``db_optimize``: Database maintenance functions.
 - ``update_profile`` / ``upload_avatar``: Admin credentials management.
-

5. Functional Conditions & Business Logic Rules

The application enforces specific business rules to maintain database and inventory integrity:

1. Inventory Availability Check: A book cannot be borrowed if `available_quantity <= 0`.
2. Borrow Limit Check: Users cannot exceed the maximum borrow count defined in Settings.
3. Overdue Block Check: If a member has outstanding overdue books, checkout is blocked.
4. Fine Calculation Logic: Fines are computed when returning a book. If `actual_return_date > return_date`, the number of overdue days is multiplied by the fine rate defined in settings.
5. AI Offline Redirection: If Gemini API fails, the backend switches to local string matching to answer basic queries, preventing system failure.